COMPUTER ARCHITECTURE & ORGANIZATION

REGISTER TRANSFER AND MICROOPERATIONS



Prepared By Nigatu A.

3-1 REGISTER TRANSFER LANGUAGE

- A digital system is an interconnection of digital hardware modules
- The modules are constructed from such digital components *as registers, multiplexers, decoders, arithmetic elements and control logic.*
- The operations executed on data stored in registers are called *Microoperations.*
- These microoperations are elementary operations that performed on the information that stored in one or more register.
- Example of microoperations are shift, clear, count, load, increment…etc
- The internal H/W Organization of a digital computer is best defined by specifying:
  - ✓ The set of registers it contains their function.
  - ✓ The sequence of microoperations performed on the binary information stored on the registers.
  - ✓ The control that initiate the sequence of microoperations.
- It is more convenient to use symbols to describe the sequence of transfers b/n registers
- The symbolic notation used to describe the microoperation transfers among registers is called *a register transfer language.*

3.2 REGISTER TRANSFER

- *The use of symbols instead of a narrative explanation provides an organized and concise manner*
- So that, *a RTL is* convenient tool for describing the internal organization of digital computers in concise and precise manner
- Computer registers are designated by capital letters (sometimes followed by numerals) to denote the function of the register.
- For example:
  - MAR(Memory Address Register) => to hold memory address
  - PC(Program Counter) => to hold the address of next instruction
  - IR(Instruction Register) => to hold instruction code
  - DR(Data Register) => to hold memory data
  - AC(Accumulator) => general purpose processing register
  - R1(Processor Register) => Processing register
- The individual flip-flops in an n-bit register are numbered in sequence from 0 through n - 1, starting from 0 in the rightmost position increasing the numbers toward the left.

Cont'd

- Figure 3-1 shows the representation of registers in block diagram form
- Naming of the register inside is shown in (a).
- The individual bits can be distinguished as in (b)
- The numbering of bits in a 16-bit register can be marked on top of the box as shown in (c)
- A 16-bit register is partitioned into two parts in (d)
- Bits 0 through 7 are assigned the symbol L (for low byte) and bits 8 through 15 are assigned the symbol *H*(for high byte).

- The name of the 16-bit register is *PC.*
- The symbol *PC(O-7) or* PC(L) refers to the *low-order byte* and
- PC(8-15) or *PC(H) to the* high-order byte.

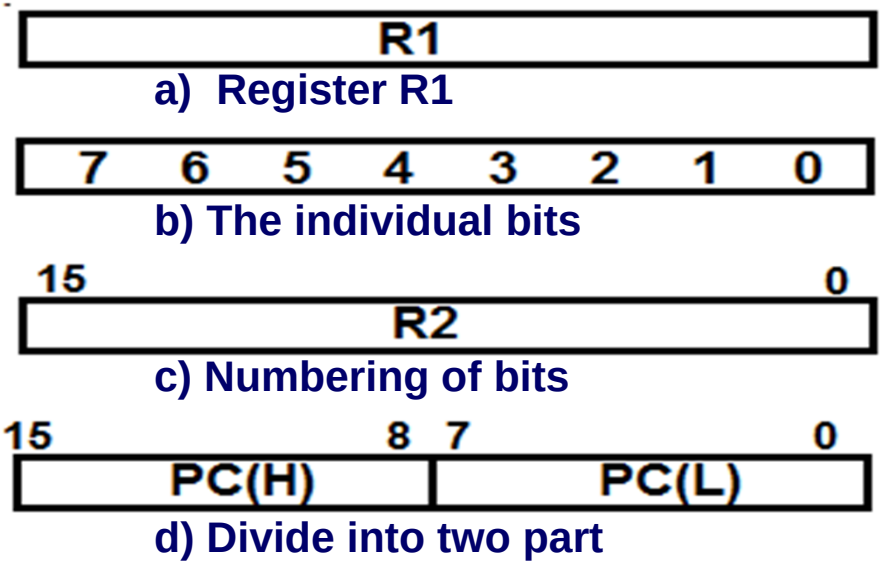


Figure 3-1

REGISTER TRANSFER(cont…)

- Transfer of the contents of one register to another is a register transfer, and can be designated in symbolic form.
- The statement : *R2 ← R1* , denotes
  - A transfer of the content of register R 1 into register *R2(*replacement of the content of R2 by the content of R1)
  - *A simultaneous transfer of all bits from the source R1 to the destination register R2, during one clock pulse*
  - The content of the source register R1 does not change after the transfer.
- A register transfer such as : *R2 ← R1,* Implies that the digital system has:
  - *The data lines from the source register (R1) to the destination register (R1)*
  - *Parallel load in the destination register (R3)*
  - *Control lines to perform the action*

CONTROL FUNCTIONS

- *Normally,* the transfer action is occur only when a *predetermined control condition is true* .
- This *can be shown by means of an if-then statement, as follow:*
  - *If(P = 1) then (R2 ← R1), where p is control signal*
- *A control function is a Boolean variable that is equal to 1 or 0.*
- *The control function is included in the statement as follows:*
  - P: R2 ← R1, *which means "if P = 1, then load the contents of register R1 into register R2",*
  - *i.e., if (P = 1) then (R2 ← R1)*
- *The control condition is terminated with a Colon.*
- Every statement written in a register transfer notation implies a hardware construction for implementing the transfer.

HARDWARE IMPLEMENTATION OF CONTROLLED TRANSFERS

- Figure 3-2 shows the Implementation of controlled transfer:

  P: R2 ← R1

- The n-output of register R1 are connected to the n-inputs of register R2, where n is the number of bits of register

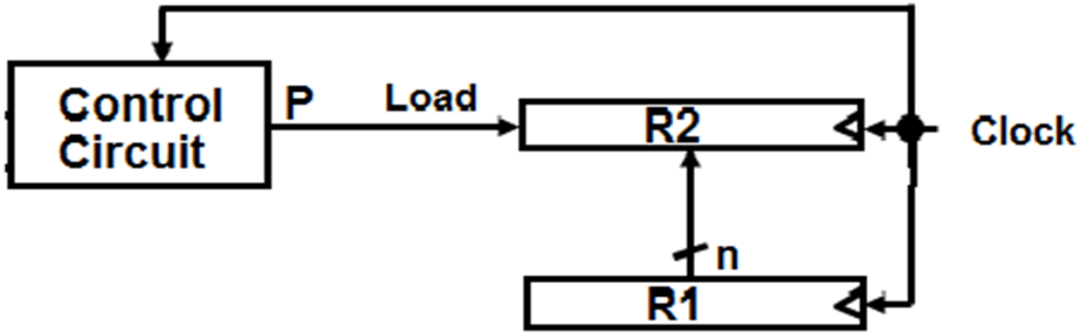


Fig.3.2 a) Block diagram of P: R2 ← R1

- Register R2 has a load input that is activated by the control variable P.
- The same clock controls the circuits that generate the control function and the two registers.
- It is assumed that the control variable is synchronized with the same clock as the one applied to the registers.

Cont'd

- The timing diagram of P: R2 ← R1 , is shown as follow:

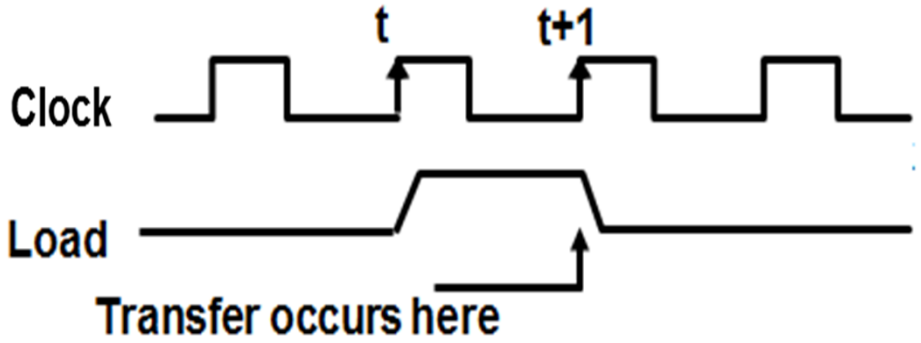


Fig3.2 b)Timing diagram of P: R2 ← R1

- P is activated in the control section by the rising edge of a clock pulse at time *t*.
- The next PET of clock at a time *t+1* finds the load i/p active and data inputs of R2 are then loaded into the register in parallel.
- Even though the control condition such as *load P* become active just after time *t*, the actual transfer does not occur until the register is triggered by the next positive transition of the clock at time *t+1*.

BASIC SYMBOLS FOR REGISTER TRANSFER

The basic symbols for register transfer notation are listed in table 3.1

Symbols	Description	Examples
Capital letters & numerals	Denotes a register	MAR, R2
Parentheses ()	Denotes a part of a register	R2(0-7), R2(L)
Arrow ←	Denotes transfer of information	R2 ← R1
Colon :	Denotes termination of control function	P:
Comma ,	Separates two micro-operations	A ← B, B ← A

- **If two or more operations are to occur simultaneously, they are separated with commas**

  P: R3 ← R5, MAR ← IR

- **Here, if the control function P = 1, load the contents of R5 into R3, and at the same time (clock), load the contents of register IR into register MAR**

3.3. BUS AND MEMORY TRANSFERS

- A typical digital computer has many registers, and paths must be provided to transfer information from one register to another.
- The number of wires will be excessive if separate lines are used b/n each register in the system.
- A more efficient scheme for transferring information b/n registers in a multiple-register configuration is a common bus system.
- Bus is a path(of a group of wires) over which information is transferred, from any of several sources to any of several destinations
- *Control signals* determine which register is selected by the bus during each particular register transfer.
- One way of constructing a common bus system is with *multiplexers.*
- The multiplexers select *the source register* whose binary information is then placed on the *Bus*.

Cont'd

- The construction *of a Bus system for four registers is shown in fig. 3.3.*

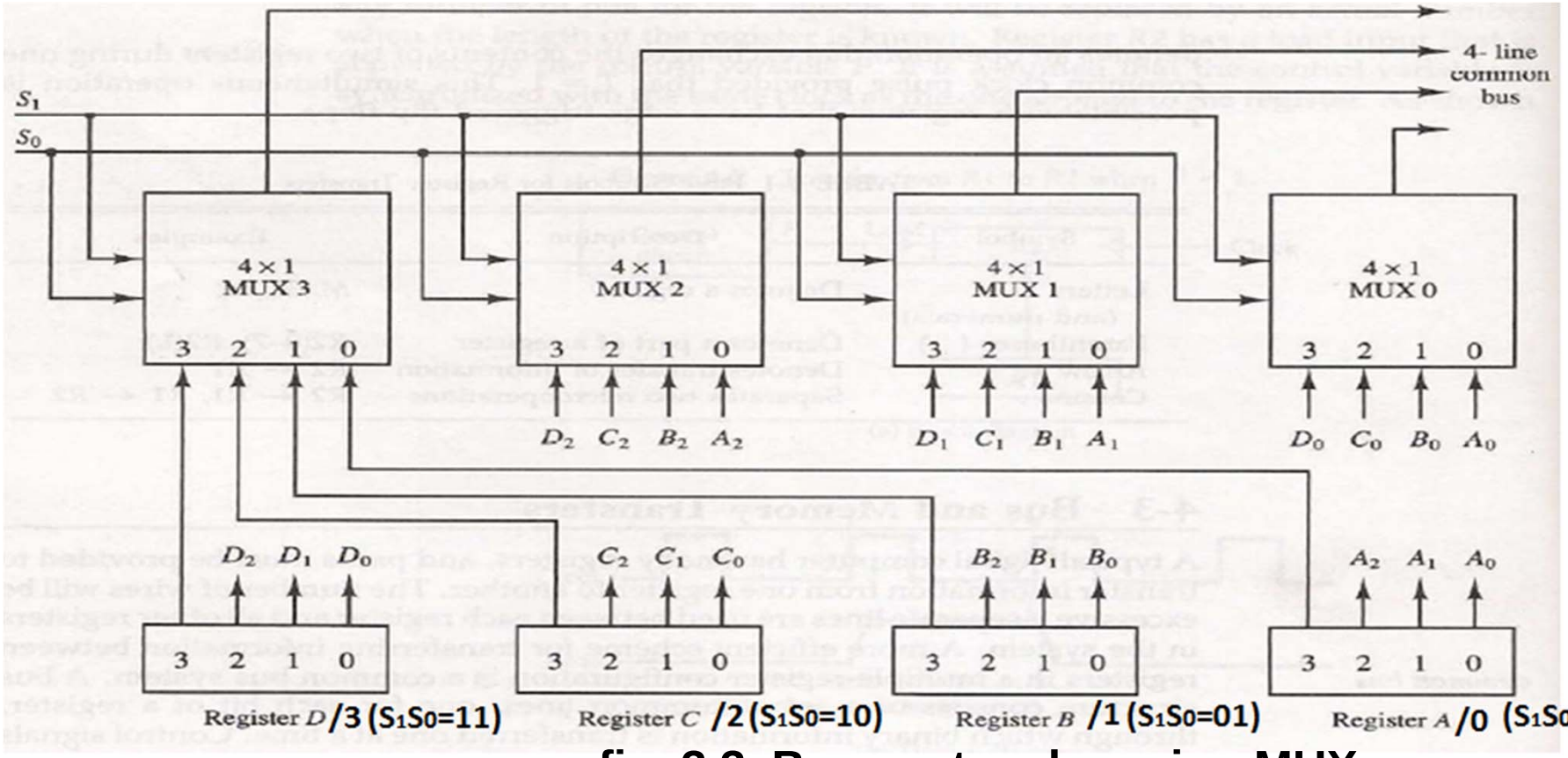


fig. 3.3. Bus system by using MUXs.

- **Each register has 4-bits, so that the bus system has 4-common lines.**
- **The bus consists of four 4x1 multiplexers each having four data inputs, and two selection inputs lines, S1 and S0.**

Cont'd

- Each register are of *4-bit* register, so we need 4-MUXs to carry the 4-bits of register *on 4-line common bus*(one line carry only one bit at a time).
- Multiplexers are to select one of four registers and transfer its content to the 4-line common bus
- The two selection lines *S1* and *S0* are connected to the selection inputs of all four multiplexers.
- The selection lines *choose* the four bits of one register and transfer them into the 4-lines common bus.
- When S1So = *00, the* 0 data inputs of all four multiplexers are selected and applied to the outputs that form the bus.
- This causes the bus lines to receive *the* content of register *A*
- *Similarly, register B is selected if S1S0 = 01, and so on.*
- Table *3-2* shows the register that is selected by the bus for each of the four possible binary value of the selection lines.

S1	S0	Register selected
0	0	A
0	1	B
1	0	C
1	1	D

Table *3-2*
Function table for bus fig3.3

Cont'd

- **The bits in the same significant position in each register are connected to the corresponding number of MUX.**
- **For example: since register A is selected by S1S0=00,**
  - **A0 is connected to input 0 of MUX 0,**
  - **A1 is connected to input 0 of MUX 1**
  - **A2 is connected to input 0 of MUX 2**
  - **A3 is connected to input 0 of MUX 3**
- **Similarly, register B is selected by S1S0=01,**
  - **B0 is connected to input 1 of MUX 0**
  - **B1 is connected to input 1 of MUX 1**
  - **B2 is connected to input 1 of MUX 2**
  - **B3 is connected to input 1 of MUX 3 and so on.**

Cont'd

- In general, a bus system with multiplex k registers of n-bits each to produce an n-line common bus.
- The number of multiplexers needed to construct the bus is equal to n, the number of bits in each register.
- The size of each multiplexer must be k x 1 since it multiplexes k data lines.
- For example:- A digital computer has a common bus system for 8 registers of 16 bits each.
  a) What size of MUXs are needed?
  b) How many MUXs are there in the Bus?
  c) How many selection inputs are there in each MUX?
- Solution:-
  a) Since there are 8 registers, we need size of 8x1 MUXs
  b) Since each register are with 16-bits , there are 16 MUXs in the Bus.
  c) Since each MUXs are with 8x1, 3 selection lines inputs in each MUXs.

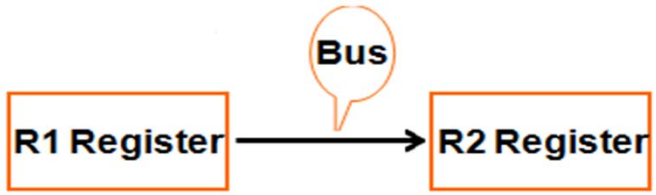
Cont'd

- The transfer of information from a bus into one of many destination registers can be accomplished by connecting the bus lines to the inputs of all destination registers and activating the load control of the particular destination register selected.
- The symbolic statement for a bus transfer may mention the bus or its presence may be implied in the statement.
- When the bus is includes in the statement, the register transfer is symbolized as follows:

  BUS ← R1, R2 ← BUS

  - The content of register *R1* is placed on the bus, and the content of the *BUS* is loaded into register *R2* by activating its load control input.
  - If the bus is known to exist in the system, it may be convenient just to show the direct transfer.

R2 ← R1



THREE STATE BUS BUFFERS

- A Bus system can be constructed with three-state *gates instead of MUXs.*
- A three-state *gate is a digital circuit that exhibits three states.*
- These states are 0, 1, High-impedance(*Open circuit*)
- Buffer is:
  - A device designed to be inserted between other devices to match impedance, to prevent mixed interactions, and *to supply additional drive or relay capability*
  - Buffer types are classified as inverting or noninverting
- Tri-state buffer gate :
  - When control input =1 : The output is enabled(output Y = input A)
  - When control input =0 : The output is disabled(output Y = high-impedance)

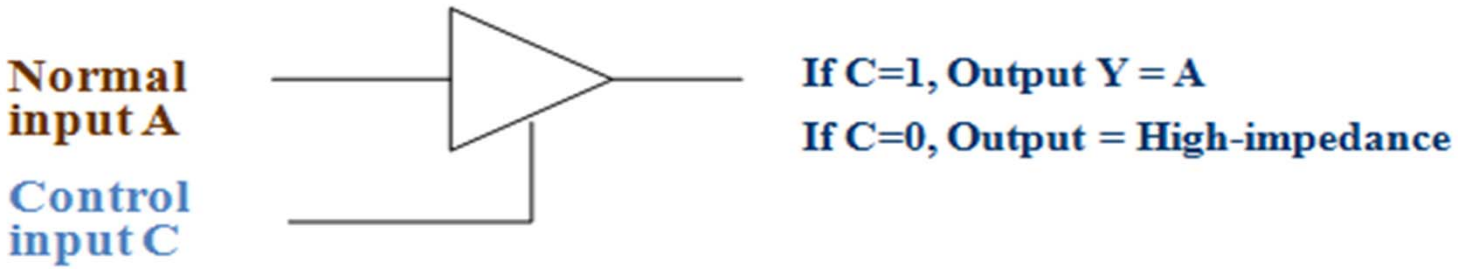


Figure 3.4 Graphic symbol for three-sate buffer.

Cont'd

- The construction of a bus system with three-state buffer is demonstrated in fig 3.5
- The outputs of four buffer are connected together to form a single bus line(Tri-state buffer
- No more than one buffer may be in the active state at any given time(the 2X 4 Decoder is to control the buffers)

- To construct a common bus for 4 register with n-bits each using three-state buffers, we need n circuits with 4 buffers in each as shown in fig. 3.5.
- Each group of 4 buffers receives one-bit from the 4 registers.
- Since each registers are with n-bits, the total common bus lines will be n.
- Example:- Draw a diagram of bus system similar to the one shown in Fig. 3-3, but use three-state buffer and a decoder instead of the MUXs.

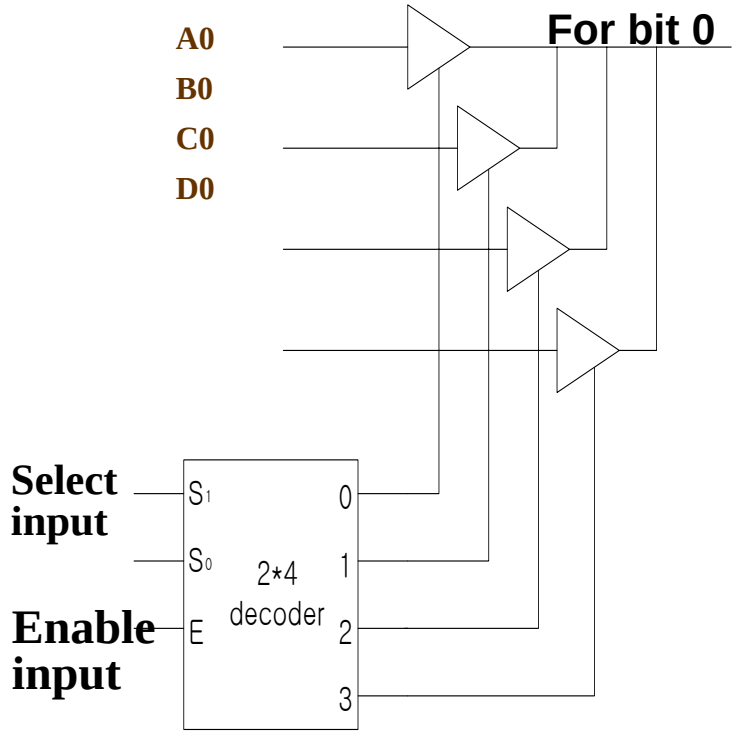


fig. 3.5. bus system using three-state buffers

MEMORY TRANSFER

- The two operations of Memory unit are Read and Write.
- Memory read :- The transfer of information from a memory word to the outside environment.
- Memory write:-The transfer of new information to be stored into the memory.
- A memory word will be symbolized by M and, Particular memory word among the many available is selected by memory address.
- This will be done by enclosing the address in square brackets following letter M (M[AR]).
- The read operation can be stated as follows:

  Read: DR ← M[AR],

  where, DR = data register, AR = address register, M[AR] = memory word
- This causes a transfer of information into *DR* from *the* memory word *M* selected by the address in AR.
- The write operation can be stated as follows: (assume that the input data are in register R1 and the address is in AR.)

  Write: M[AR]← R1
- This causes a transfer of information from R1 into *the* memory word *M* selected by the address in AR.

3.4. ARITHMETIC MICROOPERATIONS

- A microopration is an elementary operation performed with the data stored in a registers.
- The microoprations most often encountered in digital computer are classified into four categories:
  - Register transfer microoperations(transfer of binary information).
  - Arithmetic microoperations(Arithmetic operation on numeric data).
  - Logic microoperations (logical operation on non-numeric data).
  - Shift microoperations (shift operation on data stored in register)
- Register transfer microoperations was introduces in pervious sections.
- In this section we introduce a set of Arithmetic microoperations.
- The basic Arithmetic microoperations are:
  - Addition
  - Subtraction
  - Increment
  - Decrement and etc

Cont'd

- The arithmetic microoperation defined by the statement, R3 ← R1 + R2, specifies an add microoperation
- It state the Contents of R1 plus R2 transferred to R3
- The other basic arithmetic microoperations are listed in table 3.3.

Table 3.3. Summary of Typical Arithmetic Micro-Operations

R3 ← R1 + R2	Contents of R1 plus R2 transferred to R3
R3 ← R1 - R2	Contents of R1 minus R2 transferred to R3
R2 ← R2'	Complement the contents of R2
R2 ← R2'+ 1	2's complement the contents of R2 (negate)
R3 ← R1 + R2'+ 1	subtraction
R1 ← R1 + 1	Increment
R1 ← R1 - 1	Decrement

- Subtraction can be done using complementation and addition like :

  R3 ← R1 - R2 => R3 ← R1 + R2'+ 1

- The increment and decrement are symbolized by plus one and minus one operation.

BINARY ADDER / SUBTRACTOR

- The digital circuit that forms the arithmetic sum of two bits and a previous carry is called a full-adder.
- An n-bit binary adder requires n full-adders
- The n data bits for the A inputs come from register A and the n data bits for the B inputs come from B register.
- The sum can be transferred to a third register or to one of the source registers
- The addition md subtraction operations can be combined into one common circuit by including an exclusive-OR gate with each full-adder.
- A 4-bit adder-subtractor circuit is shown in fig 3.7
- The mode input M controls the operation.
- When M = 0 the circuit is an adder and when M = 1 the circuit become a subtractor

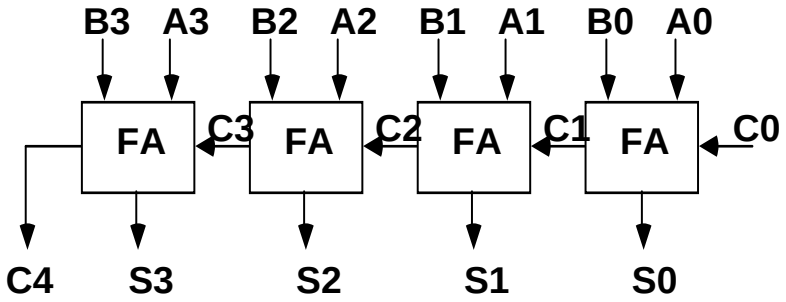


Fig. 3.6 4-bit Binary Adder

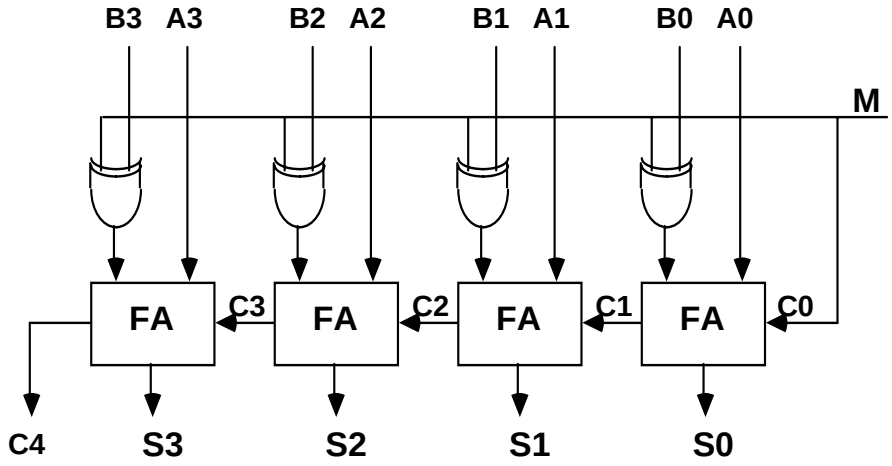


Fig 3. 7 4-bit Adder-Subtractor

BINARY INCREMENTER

- The increment microoperation adds one to a number in a register.
- This microoperation can be accomplished by means of half-adders connected in cascade.
- The diagram of a 4-bit combinational circuit incrementer is shown in Fig. 3-8.

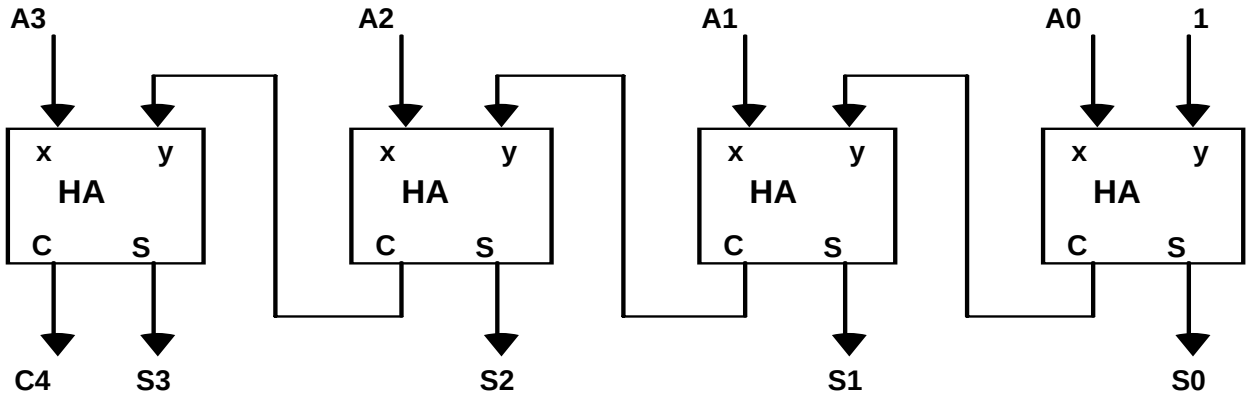


Fig.3.8
4-bit Binary Incrementer

- One of the inputs to the least significant half-adder (HA) is connected to logic 1 and the other input is connected to the least significant bit of the number to be incremented.
- The circuit of Fig. 3-8 can be extended to an n-bit binary incrementer by extending the diagram to include n half-adders.

ARITHMETIC CIRCUIT

- All the arithmetic microoperations can be implemented in one composite arithmetic circuit.
- Using parallel adder and controlling the data input to the adder, it is possible to obtain different types of arithmetic operations.
- The diagram of a 4-bit arithmetic circuit is shown in fig. 3.9.
- It has 4 FA circuit that constitute the 4-bit adder and 4 MUXs for choosing different operations.
- There are two 4-bit inputs A and B and a 4-bit output D.

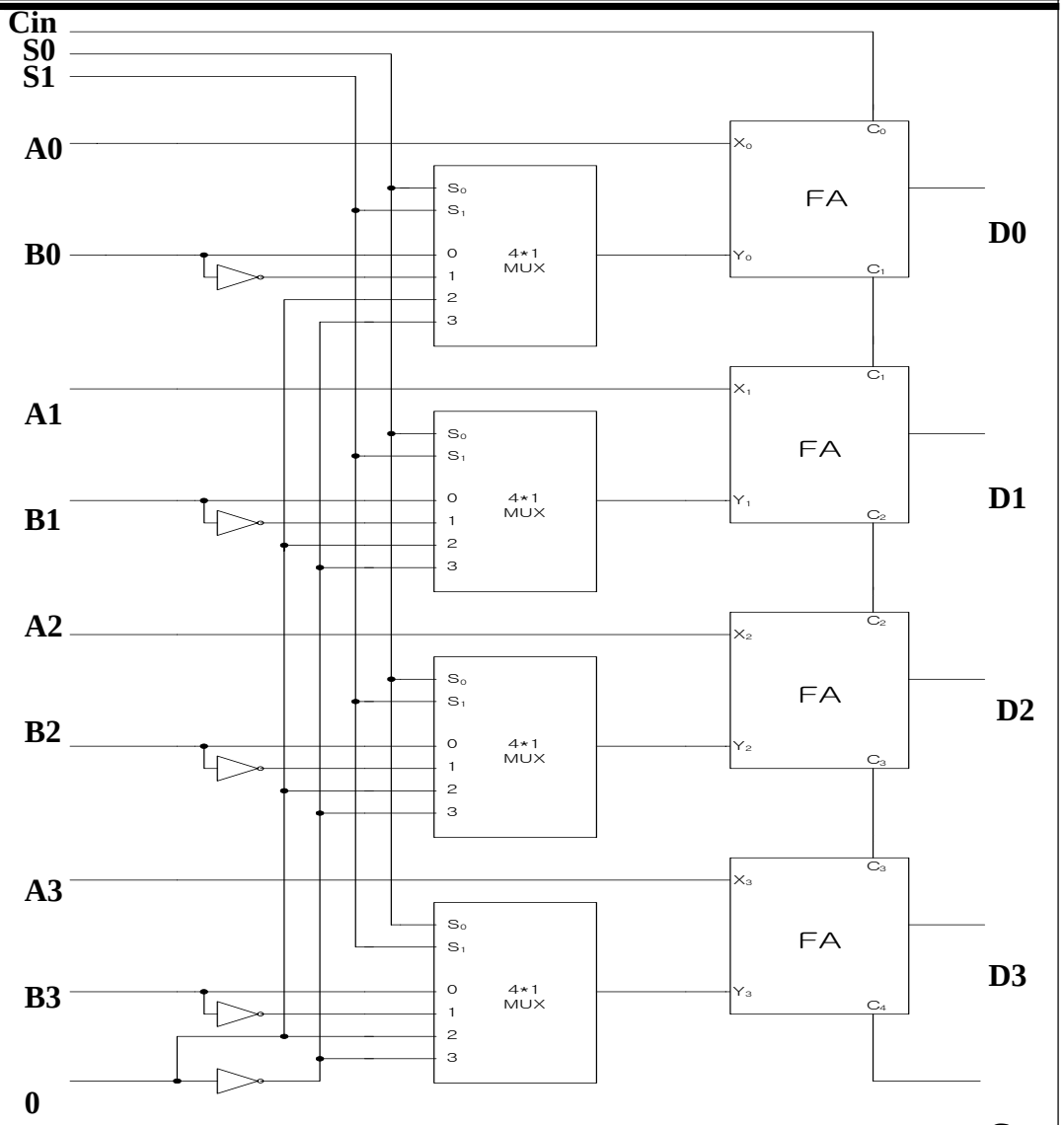


fig. 3.9 4-bit arithmetic circuit.

Cont'd

- Inputs from A go directly to the X inputs of the FA and inputs from B are connected to the data inputs of MUXs.
- The MUXs data inputs also receive the complement of B.
- The other two data inputs are connected to loggic-0 and logic-1.
- The 4 MUXs are controlled by two selection lines,S1 and S0.
- The input carry Cin goes to the carry input of the FA in the LSB.
- The output of the binary adder is calculated from the following arithmetic sum:

  $D = A + Y + Cin$,
- By controlling the value of Y with S1 and S0, and making Cin equal to 0 or 1, it is possible to generate the eight arithmetic microoperations listed in Table 3.4.

S1	S0	Cin	Y	Output	Microoperation
0	0	0	B	D = A + B	Add
0	0	1	B	D = A + B + 1	Add with carry
0	1	0	B'	D = A + B'	Subtract with borrow
0	1	1	B'	D = A + B'+ 1	Subtract
1	0	0	0	D = A	Transfer A
1	0	1	0	D = A + 1	Increment A
1	1	0	1	D = A - 1	Decrement A
1	1	1	1	D = A	Transfer A

Table 3.4. arithmetic microoperations

3.5. LOGIC MICROOPERATIONS

- Logic microoperations are bit-wise operations, i.e., they work on the individual bits of data
- These operations consider each bit of the register separately and treat them as binary variables.
- For example, the exclusive-OR microoperation with the contents of two registers R1 and R2 is symbolized by the statement :

  $P : R1 \leftarrow R1 \oplus R2$

- Let, R1 =1010 and R2 =1100 , the exclusive-OR logic computation will be done bit-by-bit on pairs of bits as following:

```
  1010   Content of R1
⊕ 1100   Content of R2
  0110   Content of R1 after P=1
```

- Logic microoperations useful for bit manipulations on binary data and for making logical decisions based on the bit value

Cont'd

- Special symbols will be adopted for the logic microoperations
  - ***OR ($\vee$),***
  - ***AND ($\wedge$),*** and
  - Complement(a bar on top), to distinguish them from the corresponding symbols used to express Boolean functions
- By using different symbols, it will be possible to differentiate between logic microoperations and a control(or Boolean) function.
- The reason is to distinguish the symbol " + ", when used to symbolized arithmetic plus or logic OR operation.
- When the symbol + occurs in a microoperation, it denote arithmetic plus and when it occurs in a control function, it denote logic OR operation
- We will never use " + " to symbolize an OR microoperation
- E.g.:- P + Q: R1 $\leftarrow$ R2 + R3 , $R4 \leftarrow R5 \vee R6$

Logic OR | Arithmetic ADD | Logic OR

LIST OF LOGIC MICROOPERATIONS

- There are, in principle, 16 different logic operations that can be defined over two binary input variables(X and Y)
- Truth Table for 16 logic functions for 2 variables is shown as below : Tab. 3-5

x	y	F_0	F_1	F_2	F_3	F_4	F_5	F_6	F_7	F_8	F_9	F_{10}	F_{11}	F_{12}	F_{13}	F_{14}	F_{15}
0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1
0	1	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1
1	0	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1

- In this table, each of the 16 columns F_0 through F_{15} represents a truth table of one possible Boolean function for the two variables X and Y.
- The 16 Boolean functions of two variables x and y are expressed in algebraic form in the first column of table 3.6.
- The corresponding 16 logic micro-operations are derived from these function by replacing variable x by the binary content of register A and variable y by binary content of register B.

Cont'd

The corresponding 16 logic micro-operations are listed as below

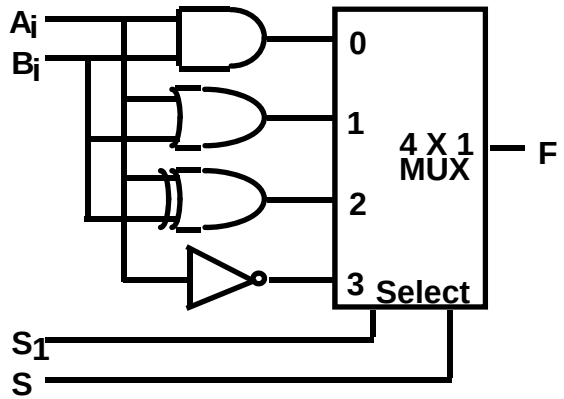
Boolean Function	Micro-Operations	Name
F0 = 0	F ← 0	Clear
F1 = xy	F ← A ∧ B	AND
F2 = xy'	F ← A ∧ B'	
F3 = x	F ← A	Transfer A
F4 = x'y	F ← A' ∧ B	
F5 = y	F ← B	Transfer B
F6 = x ⊕ y	F ← A ⊕ B	Exclusive-OR
F7 = x + y	F ← A ∨ B	OR
F8 = (x + y)'	F ← (A ∨ B)'	NOR
F9 = (x ⊕ y)'	F ← (A ⊕ B)'	Exclusive-NOR
F10 = y'	F ← B'	Complement B
F11 = x + y'	F ← A ∨ B	
F12 = x'	F ← A'	Complement A
F13 = x' + y	F ← A' ∨ B	
F14 = (xy)'	F ← (A ∧ B)'	NAND
F15 = 1	F ← all 1's	Set to all 1's

HARDWARE IMPLEMENTATION OF LOGIC MICROOPERATIONS

- Although there are I6 logic microooperations, most computers use only four- AND , OR, XOR (exclusive-OR), and complement/NOT and the others can be derived from combination of these.
- Figure 3.10 shows one stage of a circuit that generates the four basic logic microoperations.
- It consists of four gates and multiplexer.
- For a logic ckt with n-bits, the diagram must be repeated n times for I = 0 ,1,2,...n-1
- The selection lines are applied to all stages

Function table

S_1	S_0	Output	μ-operation
0	0	$F = A \wedge B$	AND
0	1	$F = A \vee B$	OR
1	0	$F = A \oplus B$	XOR
1	1	F = A'	Complement



- E X :- Draw a logic ckt for 4-bits registers

APPLICATIONS OF LOGIC MICROOPERATIONS

- Logic microoperations are very useful for manipulating individual bits or a portion of a word stored in a register
- They can be used to change bit values, delete a group of bits, or insert new bit values into a register.
- Consider the data in a register A. In another register, B, is bit data that will be used to modify the contents of A
  - Selective-set $A \leftarrow A \vee B$
  - Selective-complement $A \leftarrow A \oplus B$
  - Selective-clear $A \leftarrow A \cdot B'$
  - Mask (Delete) $A \leftarrow A \cdot B$
  - Clear $A \leftarrow A \oplus B$
  - Insert $A \leftarrow (A \cdot B) + C$
  - Compare $A \leftarrow A \oplus B$
  - . . .

Cont'd

- **Selective-set($A \leftarrow A \vee B$):-**
  - The selective-set operation sets to 1 the bits in register A where there are corresponding 1's in register B. It does not effect bit positions that have 0's in B
  - E.g. $A \leftarrow A \vee B$ (Selective-set)

    1 0 1 0 A before
    1 1 0 0 B(Logic Operand)
    1 1 1 0 A After
- **Selective-complement ($A \leftarrow A \oplus B$):-**
  - The selective-complement operation complements bits in A where there are corresponding 1's in B. It does not effect bit positions that have 0's in B
  - E.g.:- $A \leftarrow A \vee B$ (selective-complement)

    1 0 1 0 A before
    1 1 0 0 B(Logic Operand)
    0 1 1 0 A After
- **Selective-clear ($A \leftarrow A \bullet B'$):-**
  - The selective-clear operation clears to 0 the bits in A only where there are corresponding 1's in B
  - E.g.:- $A \leftarrow A \bullet B'$ (Selective-set)

    1 0 1 0 A before
    1 1 0 0 B(Logic Operand)
    0 0 1 0 A After

Cont'd

- **Selective-mask(delete) (A ← A • B):-**
  - The mask operation is similar to the selective-clear operation except that the bits of A are cleared only where there are corresponding 0's in B
  - Eg:- A ← A • B
    Selective-mask

    1 0 1 0 A before
    1 1 0 0 B(Logic Operand)
    1 0 0 0 A After
- **Insert operation:-** The insert operation inserts a new value into a group of bits
  - This is done by first masking the bits and then ORing them with the required value
  - E.g:- given A= 0110 1010, insert 1001 at the most left-four bits.

1) Mask

0110 1010 A before
0000 1111 B mask
0000 1010 A after mask



2) OR

0000 1010 A before
1001 0000 B insert
1001 1010 A after insert

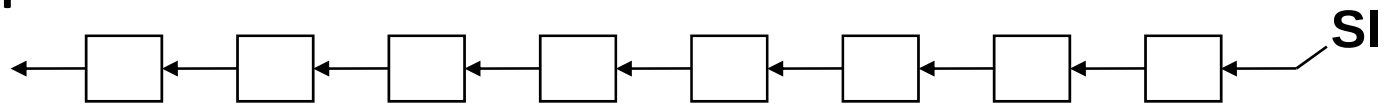
- **Clear(A ← A ⊕ B):-** The clear operation compares the words in A and B and produces an all 0's result if the two numbers are equal

e.g.:- A ← A ⊕ B

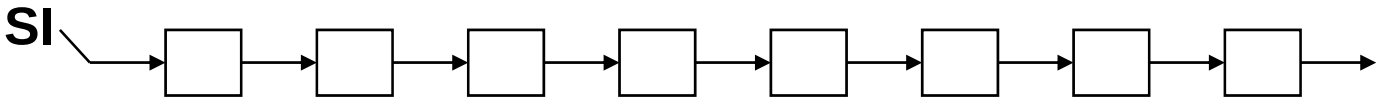
1 1 0 0 A before
1 1 0 0 B(Logic Operand)
0 0 0 0 A After clear

3.6 SHIFT MICROOPERATIONS

- Shift microoperations are used for serial transfer of data.
- The content of a register can be shifted to the right or the left.
- During a shift-left operation the serial input transfers a bit into the rightmost position.



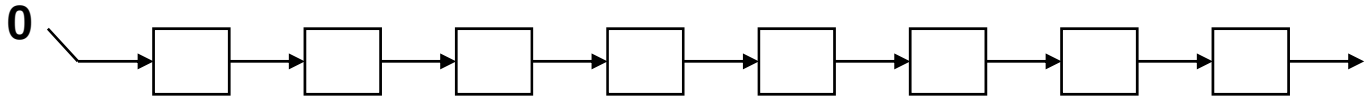
- During a shift-right operation the serial input transfers a bit into the leftmost position.



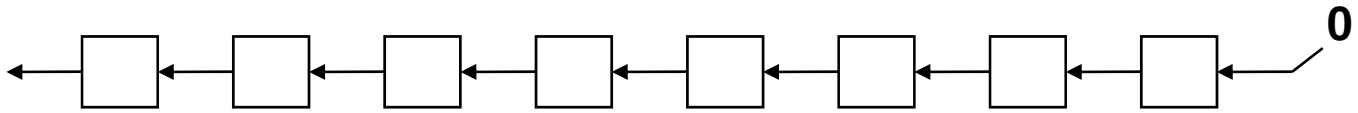
- There are three types of shifts:
  - *Logical shift*
  - *Circular shift and*
  - *Arithmetic shift*
- The information transferred through the serial input determines the type of shift.

LOGICAL SHIFT

- **A logical shift is one that transfers 0 through the serial input.**
- **We have two logical shift:**
  - **A logical right-shift operation**



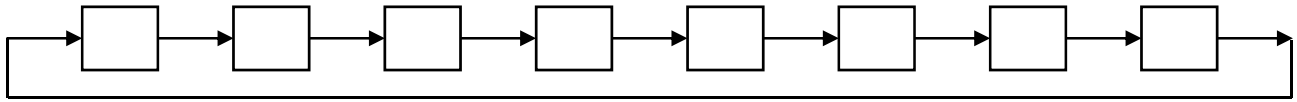
  - **A logical left-shift operation**



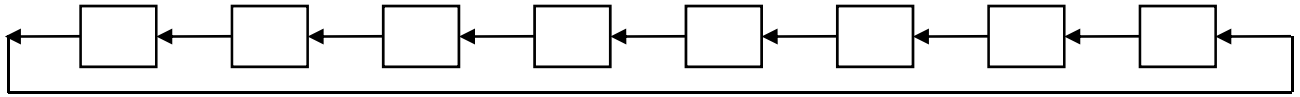
- **We will adopt the symbols *shl* and *shr* for logical shift-left and shift- right microoperations**
- **Examples: *R1 ← shr R1 , R2 ← shl R2***
  - **Specify a 1-bit shift to the right of the content of register R1 and a 1-bit shift to the left of the content of register R2**
  - **The register symbol must be the same on both sides of the arrow.**
  - **The bit transferred to the end position through the serial input is assumed to be 0 during a logical shift.**

CIRCULAR SHIFT

- The circular shift (also known as a rotate operation) circulates the bits of the register around the two ends without loss of information.
- This is accomplished by connecting the serial output of the shift register to its serial input.
- In a circular shift the serial input is the bit that is shifted out of the other end of the register.
- A right circular shift operation:



- A left circular shift operation:



- In a RTL, the following notation is used
  - ➢ *cil* for a circular shift left
  - ➢ *cir* for a circular shift right
  - ➢ Examples:
    - ✓ R2 ← *cir* R2
    - ✓ R3 ← *cil* R3

ARITHMETIC SHIFT

- *An arithmetic* shift is a microoperation that shifts a signed binary number to-the left or right.
- An arithmetic shift-left multiplies a signed binary number by 2 and An arithmetic shift-right divides the number by 2.
- Arithmetic shifts must leave the *sign bit* unchanged because the sign of the number remains the same when it is multiplied or divided by 2.
- The Leftmost bit in a register holds the sign bit, and the remaining bits hold the number.
- The sign bit is 0 for positive and 1 is for negative.
- Negative numbers are in 2's complement form.
- There are two arithmetic shift microoperations
  - Arithmetic shift-right and => *ashr*
  - Arithmetic shift-left => *ashl*

A RIGHT ARITHMETIC SHIFT OPERATION:

- Figure 3.11 shows a typical register of n bits arithmetic shift-right operation:

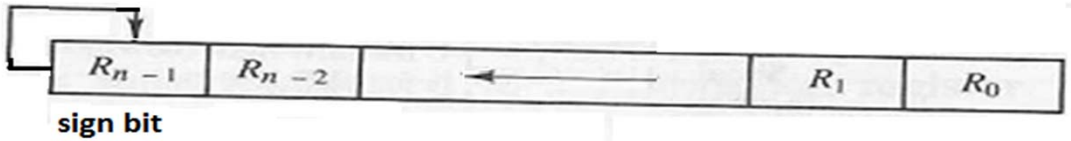


Figure 3.11 Arithmetic shift-right

- Bit R_{n-1} in the left most position holds the sign bit.
- Rn-2 is the MSB of the number and R0 is LSB of the number.
- Arithmetic shift-right leaves the sign bit unchanged and shifts the number (including the sign bit) to the right.
- Thus, R_{n-1} remains the same, R_{n-2} receives the bit from R_{n-1}, and so on for the other bits in the register, and the bit in R_0 is lost.

A LEFT ARITHMETIC SHIFT OPERATION:

- **Figure 3.11 shows a typical register of n bits arithmetic shift-left operation:**



- **The arithmetic shift-left insert a 0 into R0 , and shifts all other bits to the left.**
- **The initial bit of Rn-1 is lost and replaced by the bit from Rn-2.**
- **A sign reversal occurs if the bit in Rn-1 change in value after the shift.**
- **An *overflow* occurs after an arithmetic shift-left if initially, before the shift, Rn-1 is not equal to Rn-2.**
- **An overflow flip-flop *Vs* can be used to detect arithmetic shift-left overflow.**

$$Vs = Rn\text{-}1 \oplus Rn\text{-}2.$$

- **If Vs = 0, there is not overflow, but if Vs = 1, there is overflow and a sign reversal after the shift.**
- ***Vs* must be transferred into the overflow flip-flop.**

